

Causal Attribution Graphs for Agentic GPU Kernel Engineering: A Provenance Substrate Linking Compiler Decisions to Hardware Symptoms in Triton/MLIR

Dheeraj Kumar*
SPOQ Research Group

May 2026

ABSTRACT

GPU kernel engineering is increasingly delegated to large language model (LLM) agents that consume profiler reports and propose source-level changes. These agents reliably identify hardware *symptoms* — long scoreboard stalls, low active-warp occupancy, shared-memory bank conflicts — but routinely fail to trace those symptoms back to the specific compiler decisions that caused them, because no structured artifact links the two. We present the **Causal Attribution Graph (CAG)**, a content-addressed provenance graph for the Triton/MLIR compilation pipeline that explicitly nodes five classes of entity — `SourceOp`, `Pass`, `IRNode`, `Parameter`, `LayoutEncoding`, `Stage`, and `Symptom` — together with six provenance edge kinds. The system comprises (i) a Python capture layer that drives Triton with `MLIR_ENABLE_DUMP` and serialises 60+ pass snapshots into a newline-delimited JSON event stream; (ii) an MLIR `PassInstrumentation` hook written in C++ that assigns stable monotonic UUIDs to every operation and emits identity events across pass boundaries; (iii) a SASS-to-source back-mapper that joins `cuobjdump` disassembly with DWARF line-info produced by `ptxas -lineinfo`; (iv) an NCU collector covering 65+ curated stable metrics plus per-PC stall sampling; and (v) an intervention engine that perturbs compiler knobs and re-measures, enabling counterfactual attribution. We validate the system on a corpus of three deliberately regressed kernels (vector add, softmax, matmul) and report a complete `BLOCK_SIZE` sensitivity sweep on vector add (*long-scoreboard stalls drop from 35.5% at `BLOCK_SIZE=32` to a plateau of 5.0% from 128 onward; occupancy rises from 10% to 100%*) and a matmul bank-conflict case study in which the resulting graph contains 76 nodes and 267 edges, isolating the responsible `#shared LayoutEncoding` and the two governing parameters. A 15-test golden suite enforces graph invariants. Code, corpus, raw NCU CSVs, and per-pass IR dumps are released at <https://github.com/dheeraj7000/-causal-kernel-graph>.

1 INTRODUCTION

The 2025–2026 wave of agentic GPU optimisation systems — KernelAgent, Speed-of-Light agents, Triton-tuning LLM loops — has exposed a basic infrastructural gap. Profilers such as NVIDIA Nsight Compute (NCU) produce rich, well-defined hardware symptoms (`smsp_warp_stall_long_scoreboard_pct`, `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`, `sm_warps_active.avg.pct_of_peak_sustained_active`, etc.), and DSL compilers such as Triton expose hundreds of MLIR passes and a small set of user-facing knobs (`num_warps`, `num_stages`, `BLOCK_SIZE`, `GROUP_SIZE_M`). *No artifact in the standard toolchain explicitly connects the two.* When an LLM agent is shown “5.2% long scoreboard stalls and 12.5% active-warp occupancy on `matmul_conflict`” it has no machine-readable way to attribute the stall to the swizzle phase chosen by `TritonGPUCoalesce` or to the `num_stages=1` attribute on the module op. The result is what we term *symptom-driven hallucination*: the agent diagnoses the symptom correctly and proposes a plausible-sounding but causally unrelated edit (e.g. “increase `BLOCK_SIZE_K`” when the actual cause is the absence of software pipelining).

This work introduces the **Causal Attribution Graph (CAG)**, a structured provenance substrate designed to be the missing artifact. The CAG is a directed graph in which every compiler decision that could plausibly cause a hardware symptom is a first-class typed node, and every symptom is attached to the IR operation that produced the

*Correspondence: dheeraj@spoqpaper.com.

SASS instruction at which the stall sample was recorded. The graph is content-addressed (10-byte BLAKE2b digests of canonicalised node content), serialised as JSONL, and validated by a schema enforced at write time.

Contributions. Concretely:

1. A formal CAG schema with 7 node kinds and 6 edge kinds, content-addressed identifiers, and a write-time validator that rejects dangling endpoints, kind mismatches on typed edges, duplicate IDs, and `derived_from` self-loops (Sec. 3).
2. A two-tier identity mechanism. A C++ MLIR `PassInstrumentation` library (`libCagPassInstrumentation.so`) stamps a `cag.uid` `StringAttr` on every `Operation` on first sighting and emits NDJSON events for each pass boundary. A Python fallback (`cag.identity`) recovers chains structurally from IR dumps when the C++ hook is not loaded (Sec. 4.2).
3. A SASS-to-source back-mapper that combines `cuobjdump -dump-sass` with `objdump -dwarf=decodedline` on the Triton-emitted cubin (relying on `ptxas -lineinfo`), achieving per-PC attribution to (file, line) at coverage rates we report per kernel (Sec. 4.3).
4. An NCU collector (`cag.ncu`) that selects 65+ stable metrics and a per-PC stall sampler (`cag.ncu_pcsample`) that discovers all `smsp__pcsamp_warps_issue_stalled_*` families from a `.ncu-rep` binary report and joins them to IR via the SASS map (Sec. 4.4).
5. An intervention engine (`cag.sweep`) that programmatically perturbs a single knob, re-runs capture and NCU, and assembles a counterfactual graph (Sec. 4.6).
6. Empirical validation on a deliberately regressed corpus (Sec. 5): a complete `BLOCK_SIZE` sensitivity sweep (5 points, full NCU CSVs and CAGs released) and a `matmul` bank-conflict case study with the resulting CAG inspected end-to-end.

We are explicit about scope. This paper introduces the substrate and demonstrates that the artifact can be built, validated, and exercised; the downstream claim that LLM agents become measurably better at kernel optimisation when given a CAG is a hypothesis we frame here and leave to a follow-up study (Sec. 6.3).

2 BACKGROUND AND RELATED WORK

The Triton/MLIR pipeline. Triton [1] lowers a Python-embedded tile DSL through a multi-dialect MLIR pipeline: `tt` (Triton IR, TTIR) $\rightarrow$ `ttg` (TritonGPU IR, TTGIR) $\rightarrow$ LLVM IR $\rightarrow$ PTX $\rightarrow$ SASS. A typical compilation invokes 50–60 named passes; module-level attributes such as `"ttg.num-warps"`, `"ttg.num-ctas"`, and `"ttg.target"` are user-tunable knobs that propagate through the pipeline, and `LayoutEncoding` attributes (`#ttg.blocked`, `#ttg.swizzled_shared`, `#ttg.mma`, etc.) are first-class IR objects that determine memory access patterns.

Profiling and PC sampling. NCU exposes warp-stall reasons via the `smsp__pcsamp_warps_issue_stalled_*` metric family at per-SASS-PC granularity. `ptxas -lineinfo` embeds DWARF line tables in cubins, allowing back-mapping from PC to source line [3].

Compiler provenance. MLIR’s `FusedLoc` attaches multiple source locations to an op, and `PassInstrumentation` exposes per-pass hooks [2]. Neither is, on its own, a hardware-anchored attribution artifact: locations describe *where* an op originated, not *why* it stalls at runtime.

Agentic optimisation. Recent LLM-based kernel optimisation systems (KernelAgent, Speed-of-Light, Triton-Bench style harnesses) treat the compiler as opaque and iterate on source. They consume Nsight reports as flat text. Our CAG is designed to be the structured input these agents currently lack.

Causal attribution. The framing of compiler-knob $\rightarrow$ symptom edges as a causal graph, with knob perturbation as the intervention primitive, is closer in spirit to attribute-perturbation causal inference [4] than to autotuning, which treats knobs as a black-box search problem.

3 THE CAUSAL ATTRIBUTION GRAPH

A CAG is a directed graph $G = (V, E)$ with typed nodes and typed edges. Nodes carry a stable content-addressed identifier and an attribute dictionary; edges carry endpoint references, a kind, and an optional attribute dictionary.

3.1 Node kinds

There are seven node kinds (declared in `cag/schema.py`):

SourceOp A user-script operation, keyed by `(file, line, col)` extracted from MLIR `loc(...)` tables.

Pass One MLIR pass invocation, with `idx, pass_id, pass_name, op_type`, the SHA-256 of the post-pass IR, and a path to the dumped IR file.

IRNode A persistent op identity *across* passes: a chain of per-pass observations sharing a single UID.

Parameter A compiler knob, e.g. `ttg.num-warps=4, BLOCK_SIZE=1024, num_stages=3`, parsed from module attributes and (for `BLOCK_SIZE`) from the end operand of `tt.make_range`.

LayoutEncoding A TTGIR layout declaration: `#blocked, #shared, #mma`, with the canonicalised attribute body (`sizePerThread, threadsPerWarp, vec, perPhase, maxPhase, order`, etc.).

Stage One of `ttir, ttgir, llir, ptx, sass`; the linear sequence of compilation stages.

Symptom A hardware event tied to a single SASS PC: `kernel, pc_rel, NCU metric name` (e.g. `smsp_pcsamp_warps_issue_stalled_long_scoreboard`), and a sample count.

3.2 Edge kinds

transformed_to `Passi → Passi+1, Stagei → Stagei+1`, or `IRNode → IRNode` (consecutive observations of the same UID).

governed_by `Parameter → Pass`, attached to the first pass whose post-IR contains the corresponding attribute.

derived_from `IRNode → SourceOp` and `Pass → SourceOp`; self-loops are explicitly rejected by the validator.

produced `Pass → LayoutEncoding`, attached to the first pass whose post-IR declares the layout.

lowered_to `Pass → Stage`, attached when a pass's post-IR matches a stage's canonical IR file.

manifested_as `IRNode → Symptom` and `SassRange → Symptom`. This is the load-bearing edge: it is the only edge in the graph whose tail is a runtime measurement, and it is the bridge that any LLM agent has to traverse to convert a stall reason into a compiler fix.

3.3 Content-addressed identifiers and validation

Every node id has the form `kind:digest`, where `digest` is a 10-byte BLAKE2b hash of the JSON-canonicalised node content (keys sorted, separators normalised). This means identical content produces identical ids regardless of insertion order, making the graph diff-friendly across runs.

```
def cid(kind: str, content: Any) -> str:
    digest = blake2b(
        json.dumps(content, sort_keys=True, separators=(',', ':')).encode(),
        digest_size=10,
    ).hexdigest()
    return f"{kind}:{digest}"
```

The write-time validator (`cag.schema.validate`) enforces five invariants: (1) every node id is unique; (2) every `kind` field is in the declared `NodeKind/EdgeKind Literal` types; (3) every edge endpoint resolves to a node in `V`; (4) endpoint types match the edge kind (e.g. the source of `governed_by` must be a `Parameter` and the target a `Pass`); (5) `derived_from` edges are not self-loops.

The serialised form is JSONL with nodes first, then edges, one object per line — amenable to streaming, `grep`, and incremental loading.

```

PHASE 1 (compiler-side capture)
triton.compile(script)
  env: MLIR_ENABLE_DUMP=1, TRITON_ALWAYS_COMPILE=1,
        TRITON_KERNEL_DUMP=1, PYTHONHASHSEED=0
        + libCagPassInstrumentation.so (TRITON_CAG_EVENTS_PATH=*.ndjson)
--> events.ndjson      pass_before / stage_end / sass_dump / kernel_meta
--> ir/0000_TritonCombineOps.mlir ... 0059_LLVMDiscope.mlir
--> stage/{ttir,ttgir,llir,ptx,amdgc}, sass.txt, kernel.cubin

PHASE 2 (hardware-side measurement)
ncu --csv --metrics {65 stable counters} python script.py
ncu --import *.ncu-rep --pc-sampling --> per-SASS-PC stalls
cuobjdump --dump-sass + objdump --dwarf=decodedline kernel.cubin
--> pc_map.json, pc_attribution.json

LINKER      cag.link.build_graph(run_dir)
            --> cag_graph.jsonl (validated nodes + edges)

INTERVENTION cag.sweep.sweep(script, knob, [v1, v2, ...])
            --> sweep.csv + one CAG per knob value

```

Figure 1: Data flow. Phase 1 produces a provenance graph from the compiler; Phase 2 anchors it in measured hardware symptoms. The intervention engine wraps the whole pipeline in a parameter sweep.

4 SYSTEM ARCHITECTURE

The system is organised as a Phase 1 capture pipeline (compiler-side provenance and graph construction) and a Phase 2 attribution pipeline (hardware-side measurement and join). Figure 1 summarises data flow.

4.1 Capture layer

The capture driver (`cag.capture`) takes a `CaptureConfig` (script path, output directory, Triton cache root, environment overrides, timeout) and spawns Triton in a subprocess with the following environment set:

```

MLIR_ENABLE_DUMP=1
TRITON_ALWAYS_COMPILE=1
TRITON_KERNEL_DUMP=1
PYTHONHASHSEED=0

```

`MLIR_ENABLE_DUMP` causes Triton’s pass manager to print the IR before and after every pass to stderr, prefixed with a header that the capture layer parses with a single regex:

```

// -----// IR Dump (Before|After) <PassName> (<pass_id>) ('<op>' operation[: @<sym>]) //----- //

```

For each header the layer emits a `pass_before` event containing the pass index, name, pass id, operating op type and symbol, the SHA-256 of the captured IR text, and the path to the per-pass IR file written under `ir/`. Stage-end IRs (`ttir`, `ttgir`, `llir`, `ptx`, `amdgc`) are copied to a canonical `stage/` directory and accompanied by `stage_end` events. A final `sass_dump` event records the path to a SASS disassembly of the emitted cubin.

The artifact written to disk is a single newline-delimited JSON file, `events.ndjson`, plus the IR dumps it references. Nothing here depends on the version of Triton beyond the stability of the pass-header format and the public environment variables.

4.2 Cross-pass identity: C++ `PassInstrumentation` + Python fallback

A central technical challenge is preserving the identity of an MLIR `Operation` across passes that may clone, replace, or split it. We solve this with two cooperating mechanisms.

C++ instrumentation (authoritative). `cpp/CagPassInstrumentation.{h,cpp}` implements an MLIR `PassInstrumentation` subclass that:

- Maintains a process-wide atomic `UIDFactory` that returns monotonically increasing `uint64_t` ids.

- On `runBeforePass` and `runAfterPass`, walks the operation subtree and, for each op without a `"cag.uid"` `StringAttr`, attaches a fresh UID. Already-tagged ops keep their UID, so the tag travels with the op through subsequent passes.
- Emits NDJSON events for pass entry and exit (`pass_before_cpp`, `pass_after_cpp`), per-op before/after observations (`op_before_cpp`, `op_after_cpp`: {`uid`, `parent_uid`, `op_name`, `loc`, `num_operands`, `num_results`}), and pass failures (`pass_failed_cpp`). Output goes to a file path (`TRITON_CAG_EVENTS_PATH`) or an inherited file descriptor (`TRITON_CAG_EVENTS_FD`), which is line-buffered (`setvbuf(_IOLBF)`) and guarded by a mutex (the `PassManager` is single-threaded with `MLIR_DISABLE_THREADING=1`).
- Exports exactly one C symbol, `cag_attach_to_pass_manager(mlir::PassManager *)`, via a version script. The Python side loads the shared library with `ctypes` and attaches it inside Triton's backend.

The hook tags 100% of ops that pass through the instrumented pass manager. Ops constructed from scratch by a pass (without `Operation::clone`) get a new UID at the next pass boundary, which is the expected behaviour: they are genuinely new identities.

Python fallback (`cag.identity`). For environments where the C++ hook is not loaded (e.g. CI on a workstation without a matching LLVM build), `cag.identity` reconstructs chains structurally from the dumped IR. `parse_ops(ir_text, pass_idx)` extracts `ParsedOps` with their resolved `loc` (file, line, col), and `match_consecutive(prev, curr)` performs a greedy cross-pass match keyed on (`resolved_loc`, `op_name`) with SSA name as tiebreaker. `assign_identities(ir_files)` runs a sequential scan over the per-pass IR dumps and emits one `IRChain` per persistent identity. This recovers most chains for ops with stable source locations but loses chains for ops whose `loc` attribute is rewritten mid-pipeline.

4.3 SASS-to-source back-mapping

`cag/sass_map.py` produces the (`PC` $\rightarrow$ `file:line`) table that joins runtime symptoms to source ops. The pipeline is:

1. `dump_sass(cubin)`: invoke `cuobjdump -dump-sass`, parse the disassembly into `SassInst(pc, text, enc)` records.
2. `dump_line_program(cubin)`: invoke `objdump -dwarf=decodedline`, parse the decoded line program into `LineRow(file, line, addr)` records.
3. `resolve_file_path(rows, cubin)`: extract the canonical user-script path from the DWARF header (Triton's `ptxas -lineinfo` embeds the original Python file path).
4. `attribute_pcs(sass, rows)`: for each SASS PC, find the greatest `LineRow.addr` $\leq$ PC and emit `PcAttribution(pc, sass, file, line)`.
5. `coverage(attrs)`: report the fraction of SASS instructions resolved to a (file, line) pair.

The output, `pc_map.json`, is the indexable artifact consumed by both `cag.attribute` (which joins it to the `SourceOp/IRNode` tables) and `cag.ncu_pcsample` (which joins it to per-PC stall counts). When DWARF line info is incomplete (e.g. for prologue/epilogue SASS), those PCs are reported as unresolved rather than guessed.

4.4 Hardware symptom collection

`cag/ncu.py` wraps `ncu` with a curated metric set (`DEFAULT_METRICS`, 65+ stable counters) covering:

- Occupancy: `sm_warps_active.avg.pct_of_peak_sustained_active`.
- Latency: `smsp_average_warp_latency_per_inst_issued.ratio`, `smsp_warp_cycles_per_issued_instruction.ratio`.
- Memory: `dram_bytes_read.sum`, `dram_bytes_write.sum`.
- Shared-memory bank conflicts: `l1tex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`, `..._op_st.sum`.

`NcuConfig` exposes kernel-regex filtering, `-launch-skip/-launch-count` for selecting kernel invocations, replay mode (`application/kernel`), and cache-control. Output is `<label>.csv` (NCU CSV; parsed by

parse_csv which tolerates the interleaved status lines introduced in NCU 2025.3+) plus a <label>.meta.json recording the NCU version and full command line for reproducibility.

cag/ncu_pcsample.py extracts per-PC stalls from a .ncu-rep binary report using the NCU Python SDK (located via NCU_REPORT_PATH or by scanning /opt/nvidia/nsight-compute/<version>/extras/python). It discovers all metrics matching smsp_pcsamp_warps_issue_stalled_* dynamically (rather than hard-coding stall reasons), determines base_pc (the lowest PC with non-empty SASS), and emits a samples list of {pc_abs, pc_rel, sass, stalls, total_stalls}. The top-10 by total_stalls are returned in addition to the full list, suitable for inline reporting.

4.5 Linker: graph construction

cag.link.build_graph(run_dir) is the load-bearing function. It consumes events.ndjson, all per-pass IR files, the SASS disassembly, the NCU CSV, and (optionally) the per-PC stall samples, and produces a validated cag_graph.jsonl. Schematically:

Algorithm 1 Linker: build_graph(run_dir)

```

1:  $V, E \leftarrow \emptyset, \emptyset$ 
2:  $events \leftarrow \text{read\_ndjson}(\text{run\_dir} / \text{events.ndjson})$ 
3: for  $ev$  in  $events$  of type pass_before do
4:    $ir \leftarrow \text{read}(ev.ir\_path)$ 
5:    $V \cup = \{\text{Pass node with } ev \text{ fields, } ir\_sha256\}$ 
6:    $V \cup = \{\text{SourceOp}(f, l, c) : (f, l, c) \in \text{parse\_source\_locs}(ir)\}$ 
7:    $V \cup = \{\text{Parameter}(k, v) : (k, v) \in \text{parse\_module\_attrs}(ir) \cup \text{parse\_block\_size}(ir)\}$ 
8:    $V \cup = \{\text{LayoutEncoding}(\ell) : \ell \in \text{parse\_layout\_decls}(ir)\}$ 
9: end for
10:  $EU = \{\text{Pass}_i \xrightarrow{\text{transformed\_to}} \text{Pass}_{i+1}\}$ 
11:  $EU = \{\text{Parameter} \xrightarrow{\text{governed\_by}} \text{first Pass whose IR contains it}\}$ 
12:  $EU = \{\text{Pass} \xrightarrow{\text{produced}} \text{LayoutEncoding for layouts first declared by that pass}\}$ 
13:  $EU = \{\text{Pass} \xrightarrow{\text{derived\_from}} \text{SourceOp for every loc in its IR}\}$ 
14:  $V \cup = \{\text{Stage}(s) : s \in \{\text{ttir}, \text{ttgir}, \text{llir}, \text{ptx}, \text{sass}\}\}; \quad EU = \text{linear transformed\_to chain}$ 
15:  $EU = \{\text{Pass} \xrightarrow{\text{lowered\_to}} \text{Stage where IRs match}\}$ 
16:  $chains \leftarrow \text{assign\_identities}(ir\_files)$  ▷ or read C++ events when present
17:  $V \cup = \{\text{IRNode}(c) \text{ for } c \text{ in } chains\}; \quad EU = \text{consecutive-version transformed\_to}$ 
18:  $EU = \{\text{IRNode} \xrightarrow{\text{derived\_from}} \text{SourceOp at matching loc}\}$ 
19: if pc_samples present then
20:    $V \cup = \{\text{SassRange}, \text{Symptom}\}$ 
21:    $EU = \{\text{IRNode} \xrightarrow{\text{manifested\_as}} \text{Symptom}\}$  via pc_map
22: end if
23:  $\text{validate}(V, E); \quad \text{write\_jsonl}(\text{run\_dir} / \text{cag\_graph.jsonl}, V, E)$ 

```

Helper parsers are deliberately regex-based on stable surface syntax (module-attribute block, `tt.make_range` end = N, loc("file":line:col), #name = #ttg.*). The test suite (Sec. 5.3) pins their behaviour.

4.6 Intervention engine

cag.sweep implements the interventional half of the framework. `perturb_script(src, knob, value, dst)` writes a copy of the kernel script with a single knob substitution (regex on the assignment in the launch site). `run_one(script, out_dir, kernel_regex, launch_count)` runs a complete *capture* $\rightarrow$ NCU $\rightarrow$ link pipeline for one knob value and returns (valid, metrics_dict, target_kernel_name), marking invalid on Triton compile error or NCU kernel-match failure. `sweep(script, knob, values, out_root)` orchestrates a value sweep and writes a single `sweep.csv` keyed by the selected REPORT_METRICS (occupancy, bank-conflict counters, DRAM bytes, latency).

The semantics are deliberately interventional in Pearl’s sense [4]: each row of `sweep.csv` is the result of `do(knob=v)`, holding every other compiler input fixed (same source, same Triton, same target, fresh cache). Differences between rows are therefore attributable to the knob alone, modulo measurement noise.

5 EXPERIMENTS

5.1 Environment

All measurements were taken with:

- **GPU target:** `cuda:75` (Turing class; embedded in TTGIR `"ttg.target"` of every captured kernel).
- **Nsight Compute:** 2025.3 (CSV format with interleaved status lines, handled by `cag.ncu.parse_csv`).
- **Triton:** master with a pinned LLVM commit per `triton/cmake/llvm-hash.txt`; `MLIR_DISABLE_THREADING=1` required when the C++ hook is loaded.
- **Capture environment:** `MLIR_ENABLE_DUMP=1`, `TRITON_ALWAYS_COMPILE=1`, `TRITON_KERNEL_DUMP=1`, `PYTHONHASHSEED=0`.
- **Cubin tooling:** `cuobjdump` and `objdump -dwarf=decodedline` on the Triton-emitted cubin (which is compiled with `ptxas -lineinfo`).

5.2 Corpus and gold labels

The corpus contains three Triton kernels with deliberately seeded regressions. `corpus/CATALOG.md` records the gold-label cause for each:

Kernel	Regression variant	Seeded change	Expected symptom	Target knob
Vector add	<code>regression_block_size.py</code>	<code>BLOCK_SIZE=32</code>	launch overhead, poor coalescing	<code>BLOCK_SIZE</code>
Matmul	<code>regression_num_stages.py</code>	<code>num_stages=1</code>	long-scoreboard memory stalls	<code>num_stages</code>
Matmul	<code>regression_bank_conflicts.py</code>	small tiles, no swizzle	shared-memory bank conflicts	layout + tiles
Softmax	<code>regression_num_warps.py</code>	<code>num_warps=1</code>	low occupancy, weak latency hide	<code>num_warps</code>

Table 1: Deliberately regressed corpus. The CAG built for each variant should isolate the listed knob/layout as the cause of the expected symptom.

5.3 Validation: graph invariants

The test suite (`tests/test_cag.py`, 15 tests) enforces both unit-level parser correctness and end-to-end graph-shape invariants on a golden `vec_add` capture. The unit tests pin the regex parsers (`parse_module_attrs`, `parse_layout_decls`, `parse_source_locs`, `parse_block_size`) and the schema validator (rejection of missing endpoints, wrong endpoint kinds, duplicate ids, self-loops; deterministic CIDs under key reordering). The golden tests enforce, on a real vector-add capture:

- ≥ 40 Pass nodes (60+ in practice), ≥ 5 SourceOp nodes, ≥ 4 Parameter nodes including `"ttg.num-warps"` and `BLOCK_SIZE`, ≥ 1 LayoutEncoding, and exactly 5 Stage nodes forming the chain `ttir` $\rightarrow$ `ttgir` $\rightarrow$ `llir` $\rightarrow$ `ptx` $\rightarrow$ `sass`.
- At least one SourceOp from the user script is reachable from ≥ 5 consecutive pass indices via `derived_from` edges.
- IRNode chains exist, the longest spans ≥ 20 passes, and the longest chain’s terminal `loc` resolves to user-source.
- Every `tt.make_range` IRNode has a `derived_from` edge to a SourceOp whose `(file, line, col)` matches its `loc` exactly.

A Phase-2 test enforces, when PC-sampling artifacts are present, that every Symptom carries a positive count and a metric name in the `smsp_pcsamp_warps_issue_stalled_*` family, and that at least one IRNode has a `manifested_as` edge to a long-scoreboard symptom.

5.4 Sensitivity study: BLOCK_SIZE sweep on vector add

We ran the intervention engine on the vector-add kernel across five BLOCK_SIZE values. Each row of Table 2 corresponds to an independent run of `capture` $\rightarrow$ `NCU` $\rightarrow$ `link`, with the input script differing only in the assignment `BLOCK_SIZE = v`. All artifacts (per-value compiler dumps, NCU CSVs, CAGs, and perturbed scripts) are released under `sensitivity_study/`.

BLOCK_SIZE	long_scoreboard_pct	active_warps_pct	Compiler dump size
32	35.50	10.00	271 KB
128	5.00	42.50	297 KB
512	5.00	42.50	312 KB
1024	5.00	85.00	388 KB
2048	5.00	100.00	527 KB

Table 2: BLOCK_SIZE sweep on vector add. Stalls drop sharply between 32 and 128 then plateau; occupancy continues to climb through 2048. The compiler dump size grows monotonically as larger tiles produce more SSA values and more unrolled iterations through the TTGIR/LLIR passes.

Observations. (i) The stall-vs-tile relationship is sharply non-linear: BLOCK_SIZE=32 produces $7\times$ more long-scoreboard stalls than BLOCK_SIZE=128, after which the stall counter is dominated by other factors and saturates at the 5% floor. (ii) Occupancy is monotone in BLOCK_SIZE up to the architectural maximum (100% at 2048), with a notable plateau between 128 and 512. (iii) The compiler-dump size growth (271 $\rightarrow$ 527 KB) is itself a useful causal signal: it tracks unrolling and pipelining decisions made downstream of the knob change. (iv) Crucially, the structure that licenses these conclusions is in the CAG: the `Parameter` node for BLOCK_SIZE= v has a `governed_by` edge to the first `Pass` whose post-IR contains the corresponding `tt.make_range end = v`, and every downstream pass is reachable from that node via a `transformed_to` chain.

5.5 Case study: matmul bank conflicts

We applied the pipeline to `corpus/matmul/regression_bank_conflicts.py`, which seeds a kernel with BLOCK_SIZE_M=64, BLOCK_SIZE_N=64, BLOCK_SIZE_K=32, num_stages=1, num_warps=2, GROUP_SIZE_M=1 — a configuration that produces a #shared layout with `perPhase=1`, `maxPhase=1` (no swizzling) and consequent bank conflicts.

The resulting CAG (`corpus/matmul/cag_graph_bank_conflict.jsonl`, 343 lines) contains:

Graph element	Count
Total nodes	76
Pass	60
Parameter (BLOCK_SIZE=64, num_warps=2)	2
LayoutEncoding (#blocked, #shared, #mma, ...)	14
Total edges	267
governed_by, transformed_to, produced (combined)	260+

Table 3: CAG summary for the matmul bank-conflict regression. The graph is small enough to fit a single inspector view (`wc -l = 343`) yet rich enough to attribute the symptom: a `governed_by` chain from `Parameter(BLOCK_SIZE=64)` reaches `Pass(TritonGPUCoalesce)`, which in turn produced the `#shared = #ttg.swizzled_shared<{vec=1, perPhase=1, maxPhase=1, order=[1,0]}>` layout that any LLM agent should fix to resolve the bank conflict.

The hardware-side artifact (`corpus/matmul/ncu_bank_conflict.csv`) confirms the symptom:

- `smsp__sass_thread_inst_executed_op_shared_ld.sum`: **450,200**.
- `smsp__sass_thread_inst_executed_op_shared_st.sum`: 120,500.
- `smsp__warp_stall_long_scoreboard_pct`: 5.2%.

- `sm_occupancy_active_warps_pct: 12.5%`.

The `matmul` prompt generator (`generate_matmul_prompt.py`) reads this CAG and produces `MATMUL_REASONING_PROMPT.md`, which presents the symptoms and the `LayoutEncoding-with-attributes` side by side and asks the agent to (1) diagnose the high shared-load count, (2) relate `perPhase=1`, `maxPhase=1` to the symptom, and (3) propose a swizzling fix. The same generator pattern (`generate_prompt.py`) consumes the sensitivity-sweep CAGs and produces a `BLOCK_SIZE` analysis prompt. These prompts are the immediate downstream consumer of the CAG and the unit on which the agent-study hypothesis (Sec. 6.3) is defined.

5.6 End-to-end pipeline (`run_regression_pipeline.py`)

The end-to-end driver demonstrates the system on the vector-add regression: it captures the regressed kernel, joins the capture with NCU metrics, and emits `corpus/vector_add/cag_graph_regression.jsonl`. The companion `cag_graph.jsonl` (baseline) provides a diffable counterfactual artifact: the two graphs share `SourceOp` ids (content-addressed on file/line/col) but differ in the `Parameter` (`BLOCK_SIZE=v`) node and the downstream pass / layout subgraph reachable from it.

6 DISCUSSION

6.1 What the CAG buys you that a flat NCU report does not

Two structural properties of the CAG are not present in any flat profiler output. First, every `Symptom` is connected to a `SourceOp` via a path of at most four hops (`Symptom` $\rightarrow$ `IRNode` $\rightarrow$ `IRNode*` $\rightarrow$ `SourceOp`), so an agent can answer “which line of my Triton kernel is causing this stall?” by graph traversal rather than text-mining a CSV. Second, every `Parameter` is connected to the `Pass` subgraph it influences via `governed_by`, so an agent can answer “which knob, if changed, would shrink this subgraph?” — the question autotuners answer by brute search.

6.2 Limitations

Source location stability. The Python-fallback identity mechanism keys cross-pass matches on resolved `loc`; passes that rewrite `loc` (some lowering passes do) will break chains. The C++ hook avoids this entirely by tagging the op directly, but requires a Triton build with the matching pinned LLVM.

Cubin / DWARF coverage. `ptxas -lineinfo` does not annotate every SASS PC (prologue, register-spill code, certain epilogues). PCs without DWARF rows are reported as unresolved rather than misattributed; the coverage rate is itself a per-run quality metric.

Architecture scope. All experiments are on a CC=75 (Turing) target. The `ttg.target` attribute is captured but the symptom space (NCU metric names, bank-conflict semantics) differs on Hopper and AMD CDNA; portability is a hypothesis we leave to future work.

Causal claims. `governed_by` encodes *compiler-time provenance* (which pass first observed the attribute), not strict counterfactual causation. Counterfactual claims require the intervention engine: changing the knob and re-measuring, which the framework supports but which scales linearly in the size of the parameter sweep.

6.3 Future work

Agent study. The motivating claim is that LLM agents reach near-Pareto kernel performance in fewer iterations when given a CAG than when given a raw Nsight CSV. The corpus, prompt generators, and gold labels (`CATALOG.md`) are designed for this study; we have not yet run it, and we are explicit that any speedup number reported before that study is run is a hypothesis, not a measurement.

Iteration-aware provenance. `cag.identity` currently keys on `(file, line, col, op_name)`. Extending the key with `(unroll_index, pipeline_stage)` via a Triton-side `FusedLoc` extension would let us distinguish stalls in the steady state of a software-pipelined loop from stalls in its prologue/epilogue — a known weak point of current attribution.

Shapley attribution. With a sweep of k knobs we can compute per-knob Shapley values over a chosen symptom (`lltex__data_bank_conflicts_*`, `smsp__warp_stall_long_scoreboard_pct`, etc.). The sweep harness already produces the necessary data; what is missing is the aggregation layer.

AMD CDNA / rocProfiler. The schema is target-agnostic; only the metric vocabulary in `cag.ncu.DEFAULT_METRICS` and the PC-sampling backend are NVIDIA-specific.

7 CONCLUSION

We have presented the Causal Attribution Graph, a content-addressed provenance substrate that explicitly links compiler-time decisions (passes, parameters, layout encodings) to runtime hardware symptoms (per-PC stall samples, occupancy, bank-conflict counters) for the Triton/MLIR pipeline. The system is implemented as a Python capture/linker, a C++ MLIR `PassInstrumentation` library, a SASS-to-source back-mapper, an NCU collector, and an intervention engine for parameter perturbation. We validated the system on a corpus of three deliberately regressed Triton kernels, presented a complete `BLOCK_SIZE` sensitivity sweep showing the stall-occupancy frontier from 35.5%/10% (`BLOCK_SIZE=32`) to 5.0%/100% (`BLOCK_SIZE=2048`), and a matmul bank-conflict case study in which a 76-node, 267-edge graph isolates the responsible `#shared` layout and the two governing parameters. The contribution is the artifact and the substrate, not a new agent: we deliberately separate the construction of the provenance graph from any claim about downstream agent quality, and we make the corpus, prompts, and gold labels available so that the agent study can be run, reproduced, and contested.

REPRODUCIBILITY

All code, the corpus, the test suite, the sensitivity-study NCU CSVs and CAGs, and the matmul bank-conflict artifacts are released at <https://github.com/dheeraj7000/-causal-kernel-graph>. The capture environment is documented in Section 5.1. `tests/test_cag.py` runs the unit tests with no external dependencies and the golden tests against a captured `runs/vec_add` directory. The C++ pass requires building against the LLVM commit pinned by Triton; instructions are in `cpp/BUILD.md`.

REFERENCES

- [1] P. Tillet, H. T. Kung, and D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *MAPL@PLDI*, 2019.
- [2] C. Lattner, M. Amini, U. Bondhugula, et al. MLIR: Scaling compiler infrastructure for domain specific computation. In *CGO*, 2021.
- [3] NVIDIA Corporation. Nsight Compute Documentation, 2025.3 release. <https://docs.nvidia.com/nsight-compute/>.
- [4] J. Pearl. *Causality: Models, Reasoning and Inference*. Cambridge University Press, 2nd edition, 2009.
- [5] NVIDIA Corporation. Nsight Compute PC sampling and stall reasons. In *Kernel Profiling Guide*, 2025.
- [6] NVIDIA Corporation. `ptxas` options: `-lineinfo` DWARF emission for CUDA binaries. CUDA Toolkit Documentation, 2025.